



Python para Iniciantes: do Console ao Script, VS Code e Ambiente Virtual

Programação, Algoritmos e Linguagens

Linguagem de Programação

Sistema formal para escrever instruções que um computador pode executar, descrevendo cálculos, transformações de dados e processos lógicos.

Algoritmo

Sequência finita de passos que descreve como resolver um problema, como calcular uma média ou classificar dados.

Programa

Implementação de um algoritmo em uma linguagem de programação que o computador pode executar.

Quando escrevemos um programa em Python, estamos transformando um algoritmo em algo que o computador pode executar.

O que é a Linguagem Python

Python é uma linguagem **interpretada**, de **alto nível** e projetada para ser simples e legível. O interpretador traduz o código para bytecode e o executa imediatamente, sem uma etapa de compilação separada feita pelo programador.

Cada comando digitado é analisado, executado e o resultado é devolvido na hora. Esse modelo favorece a experimentação e a análise exploratória de dados.

Interpretada

O interpretador executa o código diretamente, sem uma etapa de compilação feita pelo programador.

Legível

Sintaxe clara e intuitiva, próxima da linguagem natural.

Exploratória

Ideal para experimentação e análise de dados.

O Console Python

O console Python é um ambiente interativo onde comandos podem ser digitados e executados imediatamente. Ele permite testar ideias, verificar valores e explorar dados rapidamente.

Digitação

O usuário digita um comando no console Python.

Execução

O interpretador analisa e executa o comando imediatamente.

Resultado

O resultado é devolvido instantaneamente na tela.

No Visual Studio Code, o console aparece no terminal integrado ou em janelas interativas. O console é o principal meio de interação entre o usuário e a linguagem Python.

Primeiros Comandos no Console

No console Python, expressões são executadas diretamente e os resultados aparecem de imediato.

O interpretador recebe a expressão, realiza o cálculo e devolve o resultado instantaneamente.

Exemplo Básico

```
>>> 2 + 2  
4
```

O símbolo `>>>` indica o prompt do console Python.

Variáveis e Execução Sequencial

Declaração

```
x = 10  
y = 3
```

Uma variável associa um nome a um valor na memória.

Operação

```
x + y
```

Usamos as variáveis em cálculos e operações.

Resultado

```
13
```

Python retorna o resultado da expressão.

Python executa os comandos na ordem em que aparecem, e cada linha depende do estado criado pelas anteriores. Essa execução sequencial é fundamental para entender como os programas funcionam.

Programas em Python

Exemplo de Programa

```
x = 10
y = 20
z = x + y
print(z) # 30
```

Este programa implementa o algoritmo de soma de dois números e representa um modelo formal de cálculo.

No console, digitar apenas `z` já mostra o valor. Em um script `.py`, use `print()` para exibir resultados.

Um conjunto de comandos forma um programa. Cada linha é executada sequencialmente, construindo o resultado final através de passos lógicos.

Operações Aritméticas em Python

Python pode ser usado como uma calculadora científica poderosa. Essas operações formam a base de cálculos estatísticos e científicos.

Adição e Subtração

```
2 + 3  
10 - 4
```

Operações básicas de soma e diferença.

Multiplicação e Divisão

```
5 * 6 # 30  
8 / 2 # 4.0
```

Multiplicação usa `*` e divisão usa `/`. A divisão `/` sempre retorna um número decimal (float).

Potenciação

```
2 ** 3
```

O operador `**` calcula potências ($2^3 = 8$).

Divisão Inteira e Resto

```
7 // 2 # 3 (divisão inteira)  
7 % 2 # 1 (resto da divisão)
```

`//` descarta a parte fracionária e `%` devolve o resto, útil para testar se um número é par.

Vetores em Python

Listas Python

```
v = [10, 20, 30, 40]
```

```
v
```

Sequências de valores podem ser representadas como listas nativas do Python.

Vetores são fundamentais para manipulação de conjuntos de dados e operações matemáticas vetorizadas, essenciais em ciência de dados.

Vetores NumPy

```
import numpy as np
```

```
x = np.array([10, 20, 30, 40])
```

```
x
```

Para computação científica, usamos vetores do NumPy, que permitem operações matemáticas eficientes.

Funções em Python

Funções encapsulam algoritmos reutilizáveis, permitindo organizar e modularizar o código de forma eficiente.

Definindo uma Função

```
def soma(a, b):  
    return a + b
```

Chamando a Função

```
soma(3, 5)  
# Retorna: 8
```

Funções permitem escrever código uma vez e reutilizá-lo múltiplas vezes, tornando os programas mais organizados e fáceis de manter.

Parâmetros e Argumentos

Parâmetros

São os nomes definidos na declaração da função:

```
def media(x, y):  
    return (x + y) / 2
```

x e **y** são os parâmetros da função.

Esse mecanismo é essencial para criar algoritmos flexíveis que podem processar diferentes conjuntos de dados usando a mesma lógica.

Argumentos

São os valores passados na chamada da função:

```
media(10, 20)
```

10 e **20** são os argumentos fornecidos.

Scripts em Python

O que é um Script?

Um arquivo contendo um programa completo, geralmente com extensão `.py`.

Scripts são fundamentais para trabalho profissional em ciência de dados, garantindo que análises sejam documentadas e reproduzíveis ao longo do tempo.

Como Criar?

Scripts podem ser criados no VS Code e executados pelo interpretador Python.

Por que Usar?

Permitem registrar análises, reproduzir resultados e organizar projetos científicos.

Instalando o Python

Acesse o site oficial

Visite <https://www.python.org> para baixar a versão mais recente.

Execute a instalação

Siga as instruções do instalador. No Windows, marque a opção "Add python.exe to PATH". Em macOS e Linux, o Python 3 pode já estar instalado.

Baixe o instalador

Escolha a versão adequada para seu sistema operacional (Windows, macOS ou Linux).

Verifique a instalação

No terminal, digite o comando adequado ao seu sistema. O resultado deve exibir a versão instalada, por exemplo Python 3.12.

```
python --version # Windows  
python3 --version # macOS/Linux
```

Visual Studio Code para Python

O Visual Studio Code é o ambiente de desenvolvimento usado neste curso. Ele reúne editor, terminal e depurador em um só lugar e se torna o laboratório de trabalho para Python.

Editor de Código

Interface com realce de sintaxe, autocompletar e detecção de erros.

Terminal Integrado

Execute comandos Python sem sair do editor.

Integração Python

Instale a extensão Python (Microsoft) para selecionar o interpretador, depurar e executar scripts e notebooks.

O VS Code pode ser obtido em: <https://code.visualstudio.com>

Ambientes Virtuais (Environments)

O que é um Environment?

Define quais bibliotecas e qual interpretador Python um projeto utiliza, isolando dependências.

Ativando o Environment

```
.venv\Scripts\Activate.ps1 # Windows (PowerShell)  
source .venv/bin/activate # macOS/Linux
```

Depois de ativado, python e pip passam a usar as bibliotecas do próprio projeto. No VS Code, selecione esse interpretador em 'Python: Select Interpreter'.

Criando um Environment

```
python -m venv .venv
```

Este comando cria um ambiente virtual na pasta .venv do projeto.

Benefícios

Evita conflitos entre projetos e garante reprodutibilidade das análises.

Reprodutibilidade e Boas Práticas

Em ciência de dados, um resultado só é válido se puder ser reproduzido.

Use Scripts

Documente todas as análises em arquivos executáveis e versionados.

Environments Isolados

Mantenha dependências específicas para cada projeto.

Bibliotecas Explícitas

Registre todas as bibliotecas e versões utilizadas.

Código Limpo

Organize código, use nomes claros e documente funções.

Usar scripts, environments e bibliotecas explícitas garante que análises possam ser refeitas no futuro, mantendo a integridade científica do trabalho.

Erros Comuns de Iniciantes

Erros de Digitação

Nomes de variáveis incorretos, parênteses faltando ou operadores errados são muito frequentes no início.

Conflitos de Environment

Misturar projetos diferentes no mesmo environment causa conflitos de versão e comportamentos inesperados.

Aprender a ler mensagens de erro é parte fundamental do processo de aprendizagem. Os erros são oportunidades de compreender melhor como o Python funciona.

Esquecimento de Imports

Usar bibliotecas como NumPy ou Pandas sem importá-las primeiro gera erros de nome não definido.

Execução Fora de Ordem

Executar linhas de código fora de sequência pode produzir resultados errados ou erros de variável não definida.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Uma introdução essencial aos fundamentos de programação e pensamento computacional usando Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia completo das ferramentas essenciais para trabalhar com dados em Python, incluindo NumPy, Pandas e bibliotecas de visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda os fundamentos de ciência de dados construindo algoritmos e ferramentas do zero usando Python.